The Statistically Specified `s-rwalk` Implementation in MINS

Implementation specification for `mins` version `0.1.0.dev3`

August 4, 2026

Abstract

This document specifies the separate `proposal_scheme="s-rwalk"` kernel implemented by MINS. It follows the actual Python execution path, including immutable configuration, eligible survivor selection, construction of a regularized survivor covariance, Gaussian random-walk proposals, the fixed-q_0 Metropolis correction, randomized plateau handling, adaptive scale updates, resource limits, and recorded diagnostics. The existing Dynesty-style `"rwalk"` and ensemble `"en-rwalk"` implementations are outside the scope of this document and are not reinterpreted here.

Contents

1 Scope and implementation locations

This document concerns only `"s-rwalk"`. It is a scalar-chain, Gaussian-covariance Metropolis–Hastings replacement kernel. It coexists with:

- `"rwalk"`, the existing Dynesty-style ellipsoidal-ball walk; and
- `"en-rwalk"`, the split differential-evolution ensemble walk.

The implementation is divided among four modules:

- `src/mins/config.py`: public `SRWalkSettings` and the `"s-rwalk"` proposal-scheme literal;
- `src/mins/mcmc.py`: `SRWalkSampler`, regularized covariance factor, Gaussian proposal evolution, fixed-q_0 acceptance, and scale tuning;
- `src/mins/sampler.py`: controller lifetime, dispatch, resource integration, live-point replacement, and per-iteration histories;
- `src/mins/__init__.py`: public export of `SRWalkSettings`.

No external MCMC package is required. The code uses the sampler's supplied NumPy `Generator` exclusively.

2 The constrained density targeted by the walk

Let $L(\theta)$ be the likelihood, $\pi(\theta)$ the normalized prior, and $q_0(\theta)$ the fixed normalized importance density supplied as `importance_morph`. MINS defines

$$\log \Psi_0(\theta) = \log L(\theta) + \log \pi(\theta) - \log q_0(\theta). \tag{1}$$

At one nested iteration, the discarded live point defines the threshold λ. The replacement kernel must preserve

$$p_\lambda(\theta) \propto q_0(\theta)\mathbf{1}[\log \Psi_0(\theta) > \lambda]\,. \tag{2}$$

Equation (**??**) is not the ordinary nested-sampling constrained prior unless $q_0 = \pi$. The likelihood and prior determine whether a point passes the constraint through Equation (**??**); the Metropolis density ratio itself is a ratio of the fixed importance density q_0.

3 Public configuration and exact resolution

The user passes an immutable `SRWalkSettings` object:

```
SRWalkSettings(
    n_steps=25,
    scale=None,
    facc=0.5,
    covariance_shrinkage=0.1,
    covariance_jitter=1.0e-10,
)
```

For model dimension D, the run-level `SRWalkSampler` resolves the values as follows. Write S for the configured number of steps, f_0 for the resolved target acceptance fraction, ρ for covariance shrinkage, and ϵ for covariance jitter.

Field	Default or resolved value	Validation and meaning
`n_steps`	$S = 25$	Must be a positive integer. Every successful replacement attempts exactly S scalar transitions.
`scale`	$s_0 = 2.38/\sqrt{D}$ when omitted	An explicit value must be positive and finite. It replaces the initial value but remains subject to subsequent adaptation.
`facc`	$f_0 = \min(1, \max(1/S, f_{\text{input}}))$	The input must be finite. The controller clamps it to the same range used by the existing `rwalk` controller.
`covariance_shrinkage`	$\rho = 0.1$	Must be finite and lie in $[0, 1]$.
`covariance_jitter`	$\epsilon = 10^{-10}$	Must be positive and finite. It is both an additive diagonal regularizer and the deterministic eigenvalue floor.

The factor $2.38/\sqrt{D}$ is a mixing heuristic, not part of the correctness of the target density. Each call to `MINSampler.run()` constructs a fresh `SRWalkSampler`; scale adaptation does not leak from one run call to the next.

4 Threshold, starting state, and constraint ordering

4.1 The discarded point is not a chain state

The current worst point is identified from the stored `live_log_psi0` ordering, with the stored tie breaker included when the randomized plateau policy is active. That point supplies λ, but under strict ordering it satisfies equality and lies outside Equation (**??**). It is therefore explicitly excluded from the eligible start indices.

The implementation evaluates the shared constraint helper on the stored live arrays, removes the worst index, and selects one eligible survivor uniformly:

$$I_0 \sim \text{Uniform}(\mathcal{I}_{\text{eligible}}). \tag{3}$$

The complete cached state is copied from row I_0:

$$(\theta, \log L, \log \pi, \log q_0, \log \Psi_0, t)\,. \tag{4}$$

The starting state is not reevaluated. If no eligible survivor exists, the replacement fails with `insufficient_eligible_survivors`.

4.2 Strict and randomized plateau policies

Under `tie_policy="strict"`, a proposal passes when

$$\log \Psi_0(\theta') > \lambda. \tag{5}$$

Under `tie_policy="randomized_plateau"`, every state is augmented by $t \sim \mathrm{Uniform}(0,1)$. The discarded state defines (λ, t_λ), and every parameter proposal receives a fresh t'. Passage is lexicographic:

$$\log \Psi_0(\theta') > \lambda \quad \text{or} \quad \left[\log \Psi_0(\theta') = \lambda \text{ and } t' > t_\lambda\right]. \tag{6}$$

On acceptance, both θ' and t' become current. On either a constraint rejection or an MH rejection, both current values are retained.

5 Frozen survivor covariance geometry

5.1 Input survivor set

Let the complete live matrix be

$$\Theta = (\theta_1, \ldots, \theta_N)^\mathsf{T} \in \mathbb{R}^{N \times D}. \tag{7}$$

For `s-rwalk`, the covariance input excludes the discarded worst row:

$$X = \Theta_{-\mathrm{worst}} \in \mathbb{R}^{M \times D}, \qquad M = N - 1. \tag{8}$$

This differs from standard `rwalk`, whose bounding ellipsoid is built from the complete live set. The `s-rwalk` covariance is calculated once per attempted replacement and is not cached between replacements. All entries of X must be finite.

5.2 Empirical covariance and shrinkage

Let

$$\bar{x} = \frac{1}{M} \sum_{i=1}^{M} x_i, \qquad Y_i = x_i - \bar{x}. \tag{9}$$

The implementation computes

$$C = \frac{Y^\mathsf{T} Y}{\max(M-1, 1)}. \tag{10}$$

The denominator in Equation (**??**) keeps the helper defined even when only one survivor is available. It also handles $D = 1$ without a scalar covariance special case.

The regularized covariance is

$$C_{\mathrm{reg}} = (1-\rho)C + \rho\, \mathrm{diag}(C) + \epsilon I_D. \tag{11}$$

Shrinkage damps empirical cross-covariances while retaining marginal variances. The diagonal jitter ensures a positive baseline in directions that the finite live set does not resolve.

5.3 Deterministic positive-definite factor

After explicit symmetrization, the implementation computes

$$C_{\mathrm{reg}} = V \operatorname{diag}(\ell_1, \ldots, \ell_D) V^{\mathsf{T}}. \tag{12}$$

Every eigenvalue is floored deterministically:

$$\tilde{\ell}_j = \max(\ell_j, \epsilon). \tag{13}$$

The returned proposal factor is

$$L = V \operatorname{diag}(\sqrt{\tilde{\ell}_1}, \ldots, \sqrt{\tilde{\ell}_D}), \qquad LL^{\mathsf{T}} = V \operatorname{diag}(\tilde{\ell}) V^{\mathsf{T}}. \tag{14}$$

This construction supports one dimension, fewer survivors than dimensions, and rank-deficient survivor clouds. A `NumericalInvariantError` is raised only if the input, regularized matrix, eigendecomposition, or resulting factor cannot be made finite and usable.

5.4 Geometry lifetime

The factor L is frozen for all S transitions in one replacement. It is not recomputed after accepted or rejected proposals. The scale s_k used by that replacement is frozen at the same time. Only after a complete walk is returned successfully is the controller scale adapted for the next replacement.

6 One Gaussian proposal

At each transition, draw

$$z \sim \mathcal{N}(0, I_D) \tag{15}$$

and propose

$$\theta' = \theta + s_k L z. \tag{16}$$

Conditional on frozen L and s_k, the increment is zero-mean Gaussian with covariance

$$\mathrm{Cov}(\theta' - \theta) = s_k^2 LL^{\mathsf{T}}. \tag{17}$$

Thus the parameter-space proposal is symmetric:

$$Q(\theta' \mid \theta; L, s_k) = Q(\theta \mid \theta'; L, s_k). \tag{18}$$

MINS proposes in the full physical parameter space. It does not clip, reflect, wrap, or repeatedly redraw a point at prior boundaries. A point with zero prior or likelihood simply fails the active pseudo-likelihood constraint.

7 Evaluation and fixed-q_0 Metropolis decision

Every proposal is evaluated exactly once by `BatchEvaluator`, producing

$$(\theta', \log L', \log \pi', \log q_0', \log \Psi_0') \,, \tag{19}$$

where $\log q_0'$ always comes from the original fixed `importance_morph`. The adaptive scale and survivor covariance never replace this density.

First, the implementation draws the proposed tie value and tests the active constraint. A constraint failure is rejected without an MH uniform draw. If the proposal passes, symmetry in Equation (**??**) gives

$$\log \alpha = \min\bigl(0, \log q_0(\theta') - \log q_0(\theta)\bigr) \,. \tag{20}$$

The proposal is accepted when

$$\log U < \log \alpha, \qquad U \sim \mathrm{Uniform}(0,1). \tag{21}$$

Neither $\log L' - \log L$, $\log \pi' - \log \pi$, nor $\log \Psi_0' - \log \Psi_0$ is substituted for the fixed-q_0 ratio. An accepted proposal replaces every cached current field. A rejected proposal changes none of them.

8 Exact complete-walk algorithm

For one nested replacement, the implemented sequence is:

1. Validate all live-array shapes.
2. Preflight resource limits for all S proposals.
3. Build the eligible survivor indices and fail if the set is empty.
4. Delete the worst row from the frozen live matrix and construct L from Equations (**??**)–(**??**).
5. Select one eligible survivor uniformly and copy its complete cached state without reevaluation.
6. Freeze the controller's current scale s_k.
7. Repeat exactly S times:
   (a) check the wall-time deadline;
   (b) draw z, form Equation (**??**), and evaluate it once;
   (c) draw a fresh proposed tie value;
   (d) test the active constraint;
   (e) if valid, apply Equation (**??**);
   (f) on acceptance, replace the complete current state.
8. Return the actual final chain state, even when no proposal was accepted.
9. Tune the scale using the complete walk's acceptance count.

The following pseudocode emphasizes the execution ordering:

```
eligible = eligible_survivor_indices(...)
survivors = delete(live_theta, worst)
factor = covariance_factor(
    survivors,
    shrinkage=sampler.covariance_shrinkage,
    jitter=sampler.covariance_jitter,
)
current = copy(uniform_choice(eligible))
scale_used = sampler.scale
```

```

for _ in range(sampler.n_steps):
    z = rng.standard_normal(ndim)
    proposed = evaluate(
        current.theta + scale_used * factor @ z
    )
    proposed.tie = rng.random()

    if not passes_constraint(proposed):
        continue
    n_valid += 1

    log_alpha = min(
        0.0,
        proposed.log_q0 - current.log_q0,
    )
    if log(rng.random()) < log_alpha:
        current = proposed
        n_accepted += 1

sampler.record_completed_walk(
    accept=n_accepted,
    scale=scale_used,
)
return current
```

The matrix multiplication in the implementation is explicitly `factor @ z`; the proposal is therefore distributed according to the regularized covariance in Equation (**??**).

9 Adaptive scale between replacements

Let A_k and R_k be the accepted and rejected transitions recorded for one completed `s-rwalk` replacement. In the serial implementation,

$$A_k + R_k = S, \qquad f_k = \frac{A_k}{A_k + R_k}. \tag{22}$$

Here "accepted" means acceptance after both the active constraint and the fixed-q_0 MH test. Constraint failures and valid-but-MH-rejected points both contribute to R_k. The scale update is the same acceptance-target recursion used by the existing `rwalk` controller:

$$s_{k+1} = s_k \exp\left(\frac{f_k - f_0}{D f_0}\right). \tag{23}$$

Acceptance above f_0 expands the next replacement's proposal scale; acceptance below f_0 shrinks it. The scale used inside the completed walk is never changed retroactively. After Equation (**??**), the controller's acceptance and rejection counters reset to zero.

Because an explicit `scale` is an initial value, not a fixed-scale mode, Equation (**??**) also applies when the user supplies `scale`.

10 Resource limits and failure semantics

Before choosing a start or constructing covariance geometry, MINS rejects the complete replacement without new likelihood calls when:

- $S >$ `max_proposals_per_replacement`;
- the remaining run-wide likelihood budget cannot accommodate all S calls; or
- the wall-time deadline has already elapsed.

The wall-time deadline is checked again before every scalar transition. If it expires inside a walk, the attempt returns no draw, the worst live point is left unchanged, and the shortened chain is not used. The controller is tuned only after a successful complete walk. A successful replacement has

$$n_{\text{proposed}} = n_{\text{likelihood calls}} = n_{\text{completed}} = S, \tag{24}$$

where likelihood calls in this equality refer only to calls made by the replacement. Initial live-set evaluation is additional run-wide work.

11 Recorded diagnostics

For each completed `s-rwalk` replacement, `RunHistory` records:

Field	`s-rwalk` meaning
`constraint_pass_fraction`	Constraint passes divided by all S generated candidates.
`mh_acceptance_fraction`	Final MH acceptances divided by all S generated candidates, not only by constraint-passing candidates.
`mcmc_accepted`	Count of accepted transitions in the completed chain.
`mcmc_moved`	One if at least one transition was accepted, otherwise zero.
`mcmc_completed`	Exactly S.
`acceptance_fraction`	Cumulative nested replacements divided by all proposals. This retains the sampler-wide efficiency meaning and is not the MH rate.

The current adapted scale and covariance factor are controller-local state and are not added as public `RunHistory` arrays.

12 Why the kernel preserves the desired density

Conditional on the frozen survivor geometry L and scale s_k, the Gaussian proposal is symmetric by Equation (**??**). For two states inside the permitted constraint, the target ratio from Equation (**??**) is

$$\frac{p_\lambda(\theta')}{p_\lambda(\theta)} = \frac{q_0(\theta')}{q_0(\theta)}, \tag{25}$$

which yields Equation (**??**). A proposal outside the active constraint has target density zero and is rejected. The same argument applies to the augmented (θ, t) state under Equation (**??**), because the tie variable has a uniform density that cancels in the MH ratio.

The covariance and scale remain fixed for the entire chain. Covariance is recomputed and scale is adapted only between completed replacements, so the proposal mechanics do not change in response to accepted or rejected states within the active walk.

The eligible starting state is copied from a surviving live point already in the constrained target. Consequently, any finite number of valid Metropolis transitions preserves the constrained distribution. This is an invariance statement, not an independence or mixing guarantee.

13 Distinction from the existing `rwalk`

The public schemes are intentionally separate:

Property	`rwalk`	`s-rwalk`
Settings object	`RWalkSettings`	`SRWalkSettings`
Displacement	Uniform in a transformed unit ball	Multivariate Gaussian
Geometry	Single ellipsoid bounding the complete live set	Regularized covariance of survivors only
Geometry lifetime	Cached for approximately `walks * n_live` calls	Recomputed once per replacement
Default transitions	$D + 20$	25
Initial scale	1	$2.38/\sqrt{D}$
Scale adaptation	Acceptance-target recursion	The same acceptance-target recursion
MH target	Constrained fixed q_0	Constrained fixed q_0

Selecting `"s-rwalk"` does not route through `draw_rwalk_constrained`, does not use the bounding-ellipsoid cache, and does not modify the behavior of `"rwalk"`.

14 Mixing limitations

A finite-length Metropolis kernel can preserve the correct density while remaining highly correlated with its selected starting survivor. Poor mixing may appear as low constraint-pass fractions, low MH acceptance, or frequent `mcmc_moved=0` outcomes. Users should:

- compare evidence and posterior summaries across repeated complete runs and seeds;
- test sensitivity to `n_steps`, `facc`, covariance regularization, and `n_live`;
- inspect both constraint-pass and unconditional MH acceptance histories; and
- compare against independent constrained rejection when practical.

MINS does not force an accepted transition, discard unchanged outputs, select the highest-likelihood visited state, or choose a state conditional on movement. Each of those operations would alter the intended target.

15 Usage

Default geometry and scale initialization with a longer chain:

```python
from mins import MINSampler, SRWalkSettings

sampler = MINSampler(
    model=model,
    importance_morph=importance_morph,
    proposal_scheme="s-rwalk",
    srwalk_settings=SRWalkSettings(
        n_steps=50,
        facc=0.5,
    ),
    n_live=200,
    rng=42,
)

result = sampler.run(
    dlogz=0.1,
    max_proposals_per_replacement=100,
)
```

Explicit initial scale and covariance regularization:

```python
sampler = MINSampler(
    model=model,
    importance_morph=importance_morph,
    proposal_scheme="s-rwalk",
    srwalk_settings=SRWalkSettings(
        n_steps=40,
        scale=0.75,
        facc=0.4,
        covariance_shrinkage=0.2,
        covariance_jitter=1.0e-8,
    ),
    n_live=200,
    rng=42,
)
```

The second example starts at scale 0.75, then adapts that value after each completed replacement. The hard proposal limit passed to `MINSampler.run()` must be at least `n_steps`.

Useful inspection:

```python
print(result.history.constraint_pass_fraction)
print(result.history.mh_acceptance_fraction)
print(result.history.mcmc_accepted)
print(result.history.mcmc_moved)
print(result.history.mcmc_completed)
```

16 Validation coverage

The implementation is covered at three levels:

- deterministic unit tests validate setting rejection, default and explicit scale initialization, the adaptation recursion, uniform eligible starts, survivor-only frozen covariance, exact proposal and likelihood counts, all-rejected chains, and preflight failures;
- integration tests validate public sampler dispatch, histories, resource behavior, and fixed-seed reproducibility; and

- statistical tests validate stationarity for a truncated normal target, a correlated Gaussian with a non-axis-aligned constraint, and repeated end-to-end evidence runs.

The statistical tests are specifically sensitive to the fixed-q_0 MH correction: accepting every proposal merely because it passes the constraint does not preserve the tested truncated-normal distribution.

17 References and runtime citation behavior

The Gaussian random-walk Metropolis mechanism is the symmetric-proposal case of the Metropolis–Hastings algorithm. The $2.38/\sqrt{D}$ initialization is a conventional high-dimensional Gaussian-target scaling heuristic; it is not required for detailed balance. The MINS fixed-importance target and nested replacement context follow the project's mathematical contract.

At runtime, `sampler.citations` for `"s-rwalk"` currently returns the same Skilling (2006) nested-sampling method entry exposed by `"rwalk"`. The additional references below explain the general MH and scaling context; they are not additional runtime citation entries.

References

[1] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.

[2] W. K. Hastings, "Monte Carlo sampling methods using Markov chains and their applications," *Biometrika*, vol. 57, no. 1, pp. 97–109, 1970.

[3] G. O. Roberts, A. Gelman, and W. R. Gilks, "Weak convergence and optimal scaling of random walk Metropolis algorithms," *The Annals of Applied Probability*, vol. 7, no. 1, pp. 110–120, 1997.

[4] J. Skilling, "Nested sampling for general Bayesian computation," *Bayesian Analysis*, vol. 1, no. 4, pp. 833–859, 2006. https://projecteuclid.org/euclid.ba/1340370944